

Qwirkle Report

Cheniour Maxime
Cinquanta Octave

May 2020

Contents

1 Introduction	1
2 Contribution	2
3 Analysis	3
4 Students' Feedback	3
4.1 Maxime's Feedback	3
4.2 Octave's Feedback	3

1 Introduction

This time we were asked to recreate another tabletop game in python, Qwirkle. Since Qwirkle is a much more unusual and complicated game, neither Maxime or I knew the rules of the game beforehand. It consists in disposing different tiles forming lines according to two attributes the tiles have : a color and a shape. The longer the lines are the more points one gets for adding to them, and one can only play tiles sharing an attribute at once. Those peculiar rules make the game not easy to understand at first, and even harder to recreate as a program. The game was to be displayed either inside the terminal or through an interface, the latter considered as a bonus task. The work was divided into 5 tasks : the first one consisted in enabling the game to be played by two human players, the second one for it to be played by a human against an AI, the third one for the game to have different settings : the number of players, colors, shapes and identical tiles were all to be chosen by the user. The fourth one was for the game to support a save system : the game was to be exported into a file and a file could be loaded to resume a game. The fifth one was the aforementioned graphical interface.

2 Contribution

It is in this part that we will explain in detail how our code works and what each function does in general.

Our program's first function, logically called "start", introduces a set of important variables defined by user input. Those are the different game settings. The user is asked to enter the number of players, their names, as well as the amount of colors, shapes and identical tiles they wish to play with. Most of those variables are pretty straight forward : for instance the number of colors is stored in "colorsNum". A variable called "players" is also issued from this function. It stands as an array of dictionaries for each player, in which there is a value for their name, their score and their hand. This function also creates the bag of tiles, simply called "tiles", as another array of dictionaries, where each tile has a color and a shape attribute. Lastly, this function randomly distributes six tiles from the bag to each of the players' hand.

Next we have several smaller functions used to setup different game functionalities. countAttribute is used, as its name implies, to count the different attributes of the tiles in one's hand. It proceeds to count how many tiles have the same color attribute, and that color by color, and afterwards counts the shape attribute, and returns the best value out of both attribute. This function is used by another in order to determine which player begins the game according to their attribute count.

The display functions are another set of useful functions, as those are used to display the game in the python terminal. The first one titled shape returns a corresponding unicode symbol surrounded by vertical bars. The one called color returns the corresponding color, except purple which is changed to magenta and orange to white since those two colors are not available in the module we used to color characters. The one named tile returns a colored version of the corresponding symbol, using both the color and shape functions, and the python termcolor library to do the coloring. Another function is used to quickly display a player's hand, making use of the previous function to allow for a shortcut. The last of the display functions displays this time the grid. It uses two for loops in order to fully print the two-dimensional array. It also prints out two sets of vertical and horizontal numbers used as coordinates for the player to better understand the disposition of the grid and know exactly where to input his pieces.

The turn function is used to ask the player what he wishes to do with his turn, that is whether he wants to play, trade some of his tiles, save the game or load another game. Depending on the user's input, the functions will then start one of the four associated functions. The trading function first ensures the user really wants to trade in some of his tiles, then asks him how many of his tiles he wants to get rid of and which ones. The function then proceeds to move the tiles from the player's hand to the bag, and randomly pick others from the bag to fill in the hand. The save and load functions both efficiently make use of the pickle library, which allows for an easy dumping into a file of several variables, leading to both functions being very short.

As for the play function, we did not manage to make it functional by the deadline. Therefore meaning that our game is in fact not playable, as it lacks its core functionality. It also lacks the AI implementation and the scoreboard system.

3 Analysis

As mentioned above, this project is not finished. It lacks precisely what is supposed to make it a game. This is due to only one factor : time. And by that I do not mean that you gave us too little time to finish the project, that would be a lie, it was in fact very much doable. It's the time that we both allocated to this project that was insufficient. We made the mistake of underestimating the project. It was in fact far more complicated than we first expected it to be, especially managing the grid and such. We do believe we were not the only ones who made this mistake, although this is far from an excuse. We also did not provide you with a "read me" file as we thought providing an how to play instruction to a game that cannot be played would just add insult to injury. We wish to apologize to you for not finishing the project.

4 Students' Feedback

4.1 Maxime's Feedback

What should I say... The time given is enough to finish the project but the time we gave ourselves was not enough, I wasted some time making a User Interface that will never be used as I couldn't connect it to main algorithm. As for the Main program, it is possible to save the current game and load it back, Trading can also be possible, but the game is not playable. the grid is not well implemented making it impossible to place a tile. Really far from a good job, but this will make us learn to never leave a project till the last minute. nevertheless, the project was not easy. Most of the auxiliary parts are not complicated but the main part is a lot harder to code than I thought, and is where I lost myself the most. The rules are quite vague in some parts and all the possible placements made it really hard to count the points correctly, or even place a tile without breaking the rules. It was more a chore to code than anything else, which may be due to the fact I have never played a game of Qwirkle. This concludes my feedback, Thank you.

4.2 Octave's Feedback

As you read already, this project was much more poorly done than the last one because we let ourselves get cornered as we thought we could manage such a project in less time than what you gave us, which was our mistake. I am quite infuriated with myself that I made such a common mistake for me, it would be appropriate to say that I in fact never change my behaviour in regards to working. Except this time I didn't manage to pull myself out of the

quicksand by pure luck and wit, it just wasn't enough. It ended up as being quite disappointing as all this work feels in the end like it amounts to nothing. This project feels like I punched myself in the stomach. Although we can say that at least this helps me try and learn that I should stop doing everything at the last moment, as it can be catastrophic sometimes. Also now I know which tabletop game is my least favorite of all. Hopefully by the next project I will have learned to be more efficient and cautious.